Coherent Versus Compile-Time Grover Oracles: Exact Costs and Backend-Aware Selection for Period-Restricted Search

[1,2]Prasad Patil, [1]Jitendra Tembhurne, [1]Suvra Jyoti Choudhury
[1]Department of Computer Science and Engineering, Indian Institute of Information Technology Nagpur, India – 441108
{dtei24cse008, jtembhurne, schoudhury}@iiitn.ac.in
[2]Tata Consultancy Services, Nagpur, India
p.ppatil@tcs.com

***Abstract*—Quantum search needs an oracle, a circuit that recognizes the wanted inputs. A compile-time oracle is built from a list of the wanted inputs. A coherent oracle computes the function in the circuit and needs no list. The choice sets the cost of every search, yet prior work builds one kind at a time. We treat it as a compiler decision for a search that first finds a hidden symmetry. We prove a coherent design correct, derive exact costs for both, and test them on noise models of five IBM processors and on two real processors. Computing the function twice makes the coherent oracle costlier, unless a cheap test marks many inputs that no compact list describes. The winner also depends on the machine: for one synthetic cipher-like function, the coherent oracle is cheaper on an error-corrected computer, while the list wins on all five noise models. A compiler pass built on these costs chose best in over 90% of tests on those models. On both real processors every design beat guessing on a small search, and on a larger one the list and two of three coherent oracles still did, showing that the designs work, not a speed advantage.**

***Index Terms*—Quantum computing, quantum compilation, oracle synthesis, reversible logic, Grover search, Simon's algorithm, benchmarking, noisy intermediate-scale quantum hardware.**

I. Introduction

QUANTUM algorithms that combine period finding with amplitude amplification, the technique behind Grover search, appear in the quantum cryptanalysis of symmetric ciphers [1]–[4]. They work on a function f that maps an n-bit input x to an n-bit output $f(x)$. The first stage finds a hidden period s, a nonzero n-bit string with $f(x) = f(x \oplus s)$ for every x, where $\oplus$ is bitwise exclusive OR (XOR). The second stage runs Grover search [5] for an input whose output lies in a given marked set $\mathcal{M}$. Both stages need f as a quantum circuit. The search stage also needs an oracle, a circuit that flips the sign of every marked input.

There are two ways to build that oracle. A compile-time oracle is emitted by a classical program that lists the marked inputs and adds one multi-controlled phase gate per input, a gate that flips the sign of that one input. Our earlier hybrid quantum dynamic oracle adaptation (HQ-DOA) system builds its oracle this way [6]: once Simon's algorithm has found s, a classical loop adapts the oracle to it. So do the oracles of many hardware demonstrations of search [7], [8]. A coherent oracle instead evaluates f with a reversible circuit, tests whether the output lies in $\mathcal{M}$, and undoes the evaluation [9]. The first design needs the list of marked inputs. The second needs a circuit for f and a membership test for $\mathcal{M}$, but no list of inputs.

Which oracle to emit is therefore a compilation decision. It depends on what the compiler knows about f and $\mathcal{M}$ and on the machine that runs the program, much like the classical choice between a lookup table and computed logic. One twist is specific to search: a table that marks the answers already contains them. A compiler can build it only when some classical procedure can list the marked inputs, and a benchmark that uses it measures the device but not the search [10], [11]. Prior work synthesizes, verifies, or costs one kind of oracle at a time [4], [12]–[18]. The coherent oracle itself follows the standard compute, mark, and uncompute pattern [9]. What is new here is the comparison of the two representations for the same period-restricted search, a closed form that locates the regimes in which each is cheaper, and a backend-aware pass that makes the choice for a compiler.

We build a coherent oracle, the FQ-DOA design (fully quantum dynamic oracle adaptation), which performs the adaptation of HQ-DOA inside the circuit instead of in a classical loop. The term "fully quantum" refers to this adaptation only: the period s still comes from Simon's algorithm and classical linear algebra, or from the function description, and it fixes which inputs the search covers. We prove the design correct and derive its exact cost for a stated gate decomposition. The cost covers qubits, Tooli gates (controlled-controlled-NOT gates), T gates (the costly gates of an error-corrected machine), and T-depth (the number of sequential layers of T gates). We compare it with a compile-time oracle on three function families: functions that map most inputs to one default output with a few exceptions, linear functions built from controlled-NOT (CNOT) gates, and quadratic functions built from Tooli gates. We run every design on two 156-qubit IBM Heron r2 processors, `ibm_fez` and `ibm_marrakesh`, and compare them in simulation with the noise models of these devices and three others.

The coherent oracle wins only under a narrow condition, and the condition depends on the target. It evaluates f twice per iteration, so it needs fewer T gates exactly when $2T_f + T_g$,

with T_f and T_g the T-counts of f and of the membership test, is below the T-count of the compile-time marking circuit. It needs 3.97 to 7.48 times the T gates of the compile-time oracle for exception-described functions, and 1.06 times for linear ones at $n = 14$, though with 2.9 times its two-qubit gates. When a threshold test describes $\mathcal{M}$, compile-time oracles can compress the list into cubes (inputs that agree on fixed bits), Reed–Muller forms, or ane subspaces, and for linear and quadratic functions one of them is always at least as cheap. For a cipher-like round, whose two layers scatter the marked inputs, the coherent oracle needs fewer T gates in 160 of the 233 cases with $k \geq 8$. In the case we tested on five device models, the list still wins on each. On both processors every design amplified at $n = 4$, and at $n = 6$ so did the compile-time oracle and the coherent oracles of families $\mathcal{L}$ and $\mathcal{N}$, but not D3.

This paper makes four contributions.

1) *Cost model.* A closed-form cost of coherence, exact for a stated gate decomposition, and the condition under which coherence pays. For two-to-one f whose image contains $\mathcal{M}$, the coherent oracle adds $2T_f + 8|\mathcal{M}|$ T gates per iteration to a compile-time oracle with one gate per marked input. Against three stronger compile-time syntheses, a threshold test makes the coherent oracle cheaper only for a cipher-like round (Sections III-D and IV-C).
2) *Workload screening.* Two structural results that tell a compiler whether a period-first pipeline is worth running: the restriction to one input per coset saves no iterations, and for exception-described functions the period needs exponentially many Simon samples (Sections III-C and III-E).
3) *Selection pass.* A backend-aware compiler pass that weighs the classical cost of listing the marked inputs, the logical cost, and the routed cost under a calibrated ranking heuristic. On five device models and three routing seeds it picks the best candidate in 83 of 90 decisions, against 38 for a rule that minimizes gate counts. For one cipher-like round it emits the coherent oracle for an error-corrected target and the compile-time oracle for a device (Sections III-F, IV-C, and IV-G).
4) *Correctness and evaluation.* A proof that the coherent oracle, $U_f^\dagger P_\mathcal{M} U_f$, marks exactly the intended inputs of the restricted search and returns every auxiliary qubit to zero. Here U_f evaluates f and $P_\mathcal{M}$ flips the sign of outputs in $\mathcal{M}$. A matched evaluation of all designs covers compilation steps, noise, routing, five device models, and hardware runs of every design on two processors (Sections III-C and IV).

We do not claim a speedup over classical computation: a classical program can compute the marked inputs of all three families from the function description, which lets us score every run, so the families are test beds for oracle cost. We study period-restricted search because HQ-DOA builds a compile-time oracle for it, so both designs meet on one task. The cost model, the cost of listing, and the selection pass use the period only to fix the search register (Propositions 4 to 6, Section III-F), so the same reasoning applies to plain Grover search.

Section II reviews related work and states what is new relative to our earlier HQ-DOA system [6] (Section II-E). Section III defines the oracles, their costs, and the selection pass. Section IV reports simulation and hardware results, Section V discusses compilers, benchmarks, and limitations, and Section VI concludes.

II. Related Work and Prior Art

This section reviews four lines of work and then compares the closest designs, including our earlier system, in Table I. The lines are cryptanalysis that combines period finding with search, oracle synthesis, execution on current hardware, and benchmarking.

A. Period Finding and Search in Cryptanalysis

Simon's algorithm [19] finds the period of $f : \{0,1\}^n \to \{0,1\}^n$ with $O(n)$ queries when $f(x) = f(y)$ holds exactly for $y \in \{x, x \oplus s\}$, which we call Simon's condition. Grover's algorithm finds one of k marked items among N with about $(\pi/4)\sqrt{N/k}$ oracle calls [5], [20]. Kaplan et al. used Simon's algorithm against message authentication codes and authenticated encryption, and bounded the queries it needs when f has collisions beyond those forced by the period [1]. Leander and May ran Simon's algorithm inside the Grover oracle to attack the FX construction, which adds key material before and after a block cipher [2]. Bonnetain et al. removed the need to query the keyed function on a superposition of inputs [3], and Bonnetain and Jaques gave complete circuits and cost estimates for this oine variant [4]. Xia et al. analyzed the combination with every intermediate measurement deferred [21]. Our setting is simpler, which isolates the cost of the oracle: the period is found first and then fixed, and only the search oracle must be coherent.

B. Oracle Synthesis and Its Cost

A reversible circuit can compute a value, use it, and then run backward to clear its scratch space [9]. Multi-controlled gates, which act only when all their control qubits are 1, decompose into Tooli gates with a linear number of work qubits, scratch qubits that start and end in $|0\rangle$ [22]. A Tooli gate needs six CNOTs [23] and seven T gates [24]. A relative-phase Tooli, which is correct up to phases that cancel when it is undone, needs three CNOTs [25]. A logical AND gate writes the AND of two qubits into a fresh one with four T gates, and a measurement undoes it with none [26]. Measurement-based uncomputation extends this to other registers [27], and multi-controlled gates can trade work qubits for a T-depth logarithmic in the number of controls [28].

Synthesis based on lookup tables maps a logic network to a Cliord+T circuit, the usual gate set of error-corrected machines (Cliord gates such as the Hadamard gate H and CNOT, plus T gates), and trades qubits for gates [12]. Bernasconi et al. exploit autosymmetry, $f(x) = f(x \oplus \alpha)$ for every α in a vector space, to build compact networks [13]. Our

functions are autosymmetric with the space $\{0, s\}$, and we use this to shrink the search register rather than the network. Ane functions $f(x) = Ax \oplus v$, for a matrix A and a vector v, are exactly the functions whose oracles can be built from Cliord gates alone. Stab-QRAM compiles them to one CNOT per one in A in minimum depth [29]; the oracle of our linear family has this form. Unqomp synthesizes uncomputation automatically [30], Seidel et al. generate Grover oracles that encode a database [14], and VQO compiles arithmetic and predicate oracles with machine-checked proofs [15].

Fault-tolerant designs count T gates and qubits, for example in a T-count-optimized multiplier [31]. Grover oracles for key search are coherent by necessity, since the keys cannot be listed. The oracles of Jaques et al. for the Advanced Encryption Standard (AES) and the LowMC cipher encrypt under the candidate key, compare with the known ciphertext, and undo the encryption [16]. In their revised estimates for AES-128 with one plaintext, the oracle needs 54 908 T gates and the encryption 54 400; undoing the cipher needs none, since every logical AND gate is undone by measurement. In terms of Proposition 4, T_f far exceeds T_g, and measurement removes the second evaluation. Later work trades T-depth against qubits in the S-box, the substitution step of AES [17], and interlaces computation with uncomputation [18].

Table lookups of classical data reach a T-count linear in the table size [32], which bounds the cost of a compile-time oracle. These works build, optimize, or verify one kind of oracle. We compare the compile-time and coherent kinds for the same task.

C. Execution on Current Hardware

Current superconducting processors connect each qubit to a few neighbors, so the compiler inserts SWAP gates, which exchange the states of two qubits, to bring interacting qubits together, a step called routing. Qiskit [33], the toolkit we use, routes with the SABRE heuristic by default [34]. Layout synthesis tools remain far from optimal [35]. Mapping by subgraph isomorphism [36], variation-aware mapping [37], and local search [38] reduce the added gates. Dynamic circuits trade qubits for mid-circuit measurement and reset [39], which our design avoids at the cost of work qubits. Grover search has run on noisy devices with a subdivided phase flip [40], with dynamical decoupling on up to five qubits [41], and on trapped-ion and superconducting processors [7], [8].

D. Benchmarking and Hidden Oracle Costs

Benchmark suites run textbook algorithms at growing size to characterize devices [42]. Leymann and Barzen list oracle expansion among the costs that algorithm descriptions often omit [10], and Johansson and Larsson show that the query advantage of Simon's algorithm depends on how the oracle is built [11]. We quantify one such hidden cost: the gap between a compile-time oracle, built from the answers, and a coherent oracle, which computes them.

E. Comparison With Prior Designs and With HQ-DOA

Table I compares the designs closest to ours. The designs in which Simon's algorithm runs inside the Grover oracle solve a dierent task, key recovery, in which the period depends on the key guess. HQ-DOA, our compile-time baseline, and our coherent oracle fix the period first.

HQ-DOA, our earlier system [6], first runs Simon's algorithm to find s. It then adapts the Grover oracle to s on a classical computer: a loop visits the candidate inputs, tests each against a marking rule, and emits a multi-controlled phase gate for each input that passes. The oracle is dynamic because it depends on the s found at run time, and hybrid because the adaptation runs classically between the two quantum stages. HQ-DOA was evaluated in simulation and on the IBM processor `ibm_brisbane`. Its published marking rule requires $f(x) \in \mathcal{M}$ and $f(x \oplus s) \notin \mathcal{M}$, which cannot both hold when $f(x) = f(x \oplus s)$, so it marks no input; the exclusive-or form in its algorithm marks none either. Our compile-time baseline keeps HQ-DOA's construction, a classical loop and one gate per marked input, with a rule that marks the intended inputs. The coherent oracle, its proofs, the cost model, the selection pass, and the evaluation are new to this paper.

III. Method

This section defines the search task and the three function families, builds both oracles, and proves the coherent oracle correct. It then derives the cost of each oracle, the cost of coherence, and the cost of finding the period, and turns these costs into a selection pass for compilers.

A. Problem Statement and Function Families

Let $f : \{0,1\}^n \to \{0,1\}^n$ satisfy $f(x) = f(x \oplus s)$ for a nonzero period s. We write a bit string left to right as $x_0 x_1 \cdots x_{n-1}$, so x_0 is its highest-order bit, and qubit i of a register holds bit i. The inputs fall into 2^{n-1} pairs $\{x, x \oplus s\}$, which we call cosets. Let b be the position of the leftmost 1 in s. Exactly one member of each coset has bit b equal to 0. We call it the canonical representative and write $X_0 = \{x : x_b = 0\}$ for the set of canonical representatives.

An input x is marked if $f(x) \in \mathcal{M}$ for a given marked set $\mathcal{M} \subseteq \{0,1\}^n$, and the task is to return a marked input. We write k for the number of marked canonical representatives. It diers from $|\mathcal{M}|$ in general: $k = |\mathcal{M}|$ when f is two-to-one and every element of $\mathcal{M}$ has a preimage, while an output shared by many cosets counts once in $|\mathcal{M}|$ but many times in k. The period s, the count k, and the iteration count $R = \lfloor (\pi/4)\sqrt{2^{n-1}/k} \rfloor$ are fixed before the run.

Family $\mathcal{E}$ (exceptions). A function in this family has a default output δ and c exceptional cosets $C_i = \{a_i, a_i \oplus s\}$ with outputs o_i, where a_i is canonical. Every input outside the exceptional cosets maps to δ. For example, the four-bit member of our benchmark has $s = 1010$, so $b = 0$ and X_0 holds the eight inputs that begin with 0. Its exceptional cosets are $\{0000, 1010\} \mapsto 0000$ and $\{0011, 1001\} \mapsto 0011$, its default output is 1111, and $\mathcal{M} = \{0000, 0011\}$. The canonical representatives 0000 and 0011 are the $k = 2$ marked inputs.

TABLE I
COMPARISON WITH PRIOR HYBRID PERIOD-AND-SEARCH DESIGNS. n: INPUT WIDTH; c: EXCEPTIONAL COSETS; k: MARKED CANONICAL INPUTS; $\mathcal{M}$: MARKED OUTPUTS; MCZ: MULTI-CONTROLLED Z. FAMILY $\mathcal{E}$: FUNCTIONS DESCRIBED BY A DEFAULT OUTPUT AND EXCEPTIONAL COSETS (SECTION III-A). N/A: NOT APPLICABLE.

Dimension	Simon inside Grover [2]–[4], [21]	HQ-DOA [6]	Compile-time oracle (CT, our baseline)	Coherent oracle (FQ-DOA, this work)
Task	Recover a cipher key; Simon's algorithm runs inside the Grover oracle, once per key guess	Find x with $f(x) \in \mathcal{M}$; Simon's algorithm runs once, before the search	Same as HQ-DOA	Same as HQ-DOA
Marking oracle	Coherent test of the rank of Simon samples	Phase gates emitted by a classical loop; published rule marks no input	One MCZ per marked input, or per cube of them	U_f, one MCZ per element of $\mathcal{M}$ on the output register, U_f
What the compiler must know	A circuit for the keyed function	s, $\mathcal{M}$, and the candidate inputs	The k marked inputs	A circuit or structure for f, and $\mathcal{M}$
Classical work before the run	Circuit construction; classical queries in the oine variant	A loop over 2^{n-1} candidates	$O(cn)$ for family $\mathcal{E}$; a search if f is a black box	$O(cn)$ for family $\mathcal{E}$, or synthesis of U_f
When $\mathcal{M}$ changes	n/a	Re-emit the oracle	Recompute the list and re-emit k gates	Re-emit $\lvert\mathcal{M}\rvert$ marking gates; U_f unchanged
Cost model	Asymptotic; resource estimates [4]	None	Closed form, Eq. (2)	Closed form, Proposition 4
Oracle choice	Fixed by the attack	Always the emitted list	Made per target by the selection pass of Section III-F	

Family $\mathcal{L}$ (compact circuits). A function in this family is $f(x) = L\,(x \oplus x_b s)$ over GF(2), that is, with arithmetic modulo 2, where L is an invertible $n \times n$ matrix. The map $x \mapsto x \oplus x_b s$ sends both members of a coset to the canonical one, so f is two-to-one with period s, and $f(x) = f(y)$ holds only for $y \in \{x, x \oplus s\}$. Its circuit is a network of CNOT gates.

Family $\mathcal{N}$ (quadratic functions). A function in this family is $f(x) = F(x \oplus x_b s)$, where $F(z)_i = z_i \oplus \bigoplus_{(j,l,i)} z_j z_l$ sums t products $z_j z_l$ with $j < l < i$ and $b \notin \{j, l, i\}$. Each output bit adds products of earlier bits only, so F is invertible, f is two-to-one with period s, and Simon's condition holds. Its circuit is $n-1$ CNOTs and one Tooli per product, so t sets how nonlinear, and how costly in T gates, the function is.

B. Two Oracle Designs

Both oracles act on the canonical representatives and apply $(-1)^{[f(x)\in\mathcal{M}]}$, where $[\cdot]$ is 1 when its condition holds and 0 otherwise. Grover's algorithm alternates the oracle with a diusion step, a reflection about the uniform superposition of the $n-1$ search qubits [43].

The compile-time (CT) oracle is a sequence of k multi-controlled Z gates on the search qubits, one per marked input; such a gate flips the sign of the state in which all its qubits are 1. X gates (bit flips) on either side select the zeros of that input. The oracle is cheap when k is small, but the compiler must know every marked input. Marked inputs that form a cube, a set of inputs that agree on fixed bits, share one gate on those bits. By default the selection pass builds CT as a cover of disjoint cubes: two cubes that dier in one fixed bit merge, until no pair does. The two marked inputs of our benchmark instances dier in two bits and do not merge, so there the cover equals the one-gate-per-input form. Stronger syntheses (Section III-D) only make CT cheaper, so the ratios we report where CT already wins are lower bounds (Section S4).

The coherent oracle is $U_{\mathrm{FQ}} = U_f^\dagger\, P_{\mathcal{M}}\, U_f$. Here $U_f\,|x, y\rangle = |x, y \oplus f(x)\rangle$ adds $f(x)$ to an output register y, and $P_{\mathcal{M}}\,|y\rangle = (-1)^{g(y)}\,|y\rangle$ flips the sign of the outputs that pass the membership test $g(y) = [y \in \mathcal{M}]$. The design therefore needs a circuit for f and one for g, but no list of marked inputs. U_f must act correctly only on canonical inputs $x \in X_0$, since no other input enters the search register. For family $\mathcal{E}$ the oracle uses an input register x (n qubits), an output register y (n qubits), a coset flag cf (one qubit), and a work register w ($n-2$ qubits), as in Fig. 1. Its three parts are as follows.

- U_f writes δ into y with X gates. For each exceptional coset, it detects a_i with a multi-controlled X from the $n-1$ bits other than x_b onto cf. It then adds $\delta \oplus o_i$ to y with CNOTs controlled by cf and repeats the detection to clear cf. For family $\mathcal{L}$, U_f is the CNOT network that adds Lx to y; on X_0 the column of L for bit b is not needed, and no flag is used. For family $\mathcal{N}$ it adds the t products with Tooli gates.
- $P_{\mathcal{M}}$ implements g for an explicit set of outputs: for each $m \in \mathcal{M}$, a multi-controlled Z on y that selects m with X gates. Any other circuit for g, such as a comparator, fits the same slot.
- $U_f^\dagger$ clears y. Because U_f adds $f(x)$ to y modulo 2, applying it twice gives the identity, so $U_f^\dagger = U_f$.

Algorithm S1 gives the full procedure.

The FQ-DOA design gives one coherent oracle per family: we call those of families $\mathcal{L}$ and $\mathcal{N}$ Lin and Quad, and reach that of family $\mathcal{E}$, called D3, from a design variant D0 in four compilation steps. D0 evaluates f on all 2^n inputs. It detects both members of every exceptional coset with n controls and sets a membership flag, which an extra qubit in $|-\rangle$ turns into a sign flip before the flag is cleared; D0 uses $3n+1$ qubits in total. D1 detects only the canonical member and drops x_b as a control (steps O1 and O2). D2 replaces the flag and the extra qubit by the multi-controlled Z gates of $P_{\mathcal{M}}$ (step O3), which saves two qubits. D3 uses relative-phase Toolis inside each multi-controlled gate (step O4, Section III-D). All variants share state preparation, diusion, and the decomposition code,

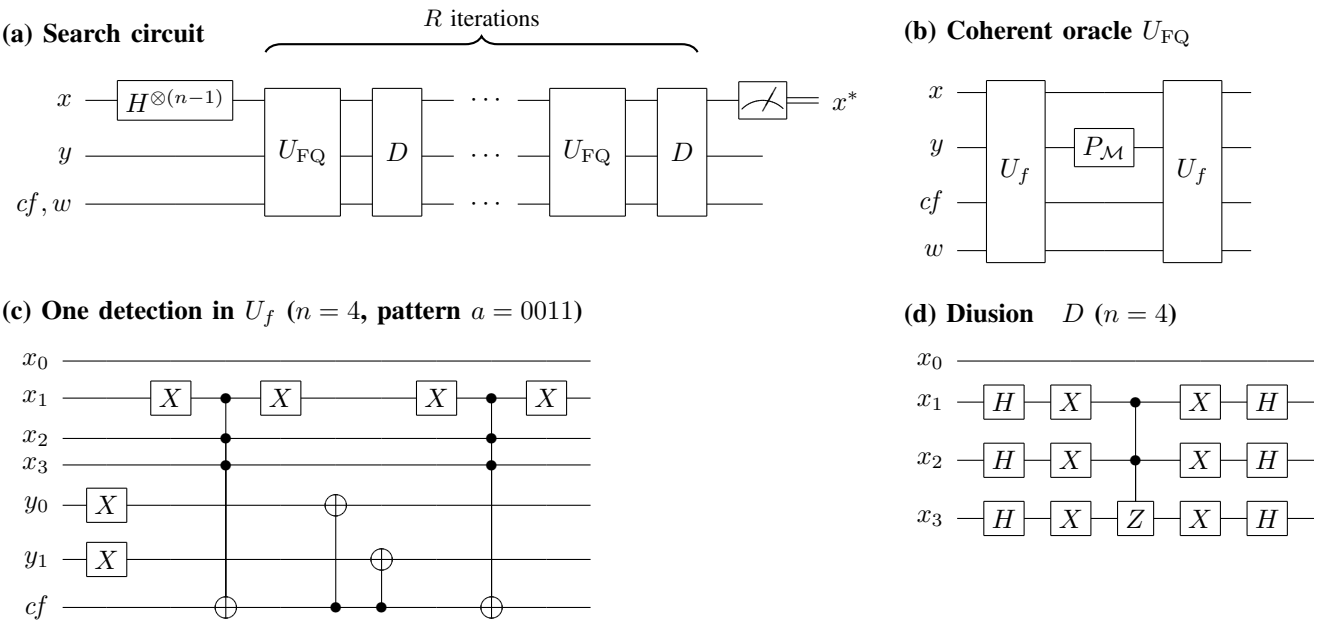


Fig. 1. Coherent-oracle search for family $\mathcal{E}$, drawn for the four-bit benchmark ($s = 1010$, representative bit $b = 0$, so $x_b = x_0$). (a) All registers start in $|0\rangle$: x and y with n qubits each, cf with one, and w with $n-2$. State preparation applies H to every input qubit except x_b, so the register holds the 2^{n-1} canonical representatives. R rounds of the oracle and diusion follow, and x is measured once at the end, giving x^*. (b) The oracle computes f into y, flips the sign of outputs in $\mathcal{M}$, and computes f again to clear y (the work register w serves the decompositions). (c) Detection of the canonical pattern $a = 0011$. The first X gates write the default $\delta = 1111$ (wires y_2, y_3 omitted). The X gate on x_1 selects the zero of a, and the multi-controlled X raises cf. CNOTs then add $\delta \oplus o = 1100$ to y, and the detection repeats to clear cf. The bit x_b is never a control. (d) Diusion acts on the $n-1$ search qubits only. No measurement occurs before the final readout.

and each step keeps the circuit correct on X_0.

C. Correctness of the Coherent Oracle

We now show that the coherent oracle flips the sign of exactly the marked canonical inputs and restores every auxiliary register. Write $|x, y\rangle$ for the input and output registers and omit cf and w when they hold $|0\rangle$.

Lemma 1: For every $x \in X_0$ and every y, $U_f |x, y\rangle = |x, y \oplus f(x)\rangle$, and cf and w return to $|0\rangle$.

Proof: For family $\mathcal{L}$ the CNOT network adds Lx to y, and $Lx = f(x)$ on X_0. For family $\mathcal{N}$ the CNOTs add x and the Tooli gates add the products, so y gains $F(x) = f(x)$ on X_0; neither network uses cf or w. For family $\mathcal{E}$, the X gates add δ to y, and the detection of a_i fires when x agrees with a_i on every bit except b. On X_0 both x_b and $(a_i)_b$ are 0, so it fires exactly when $x = a_i$, and at most one detection fires because the a_i are distinct. The CNOTs then add $\delta \oplus o_i$ if $x = a_i$, so y ends at $y \oplus o_i = y \oplus f(x)$, and otherwise at $y \oplus \delta = y \oplus f(x)$. Each detection runs twice with the same controls, which clears cf. Inside each detection, the ladder gates of the decomposition are undone in reverse order, which clears w and cancels their relative phases [25]. ∎

Lemma 2: $P_{\mathcal{M}} |y\rangle = (-1)^{[y \in \mathcal{M}]} |y\rangle$.

Proof: The multi-controlled Z for m flips the sign of $|y\rangle$ exactly when $y = m$. The elements of $\mathcal{M}$ are distinct, so at most one gate fires. ∎

Proposition 1: For every $x \in X_0$, $U_{\mathrm{FQ}} |x, 0\rangle = (-1)^{[f(x) \in \mathcal{M}]} |x, 0\rangle$.

Proof: By Lemma 1, U_f gives $|x, f(x)\rangle$. By Lemma 2, $P_{\mathcal{M}}$ multiplies it by $(-1)^{[f(x) \in \mathcal{M}]}$. By Lemma 1 again, $U_f^\dagger = U_f$ gives $|x, f(x) \oplus f(x)\rangle = |x, 0\rangle$. ∎

Proposition 2: After R iterations, the probability of measuring a marked input is $\sin^2((2R+1)\theta)$, where $\sin^2 \theta = k/2^{n-1}$.

Proof: State preparation gives the uniform superposition over X_0. No gate changes x_b, so the state stays in the span of X_0. On that span, Proposition 1 makes U_{FQ} the standard phase oracle, and diusion reflects about the uniform state. The claim is then the usual analysis of Grover's algorithm with k marked items among 2^{n-1} [20]. ∎

Proposition 2 also holds for the compile-time oracle, whose gates mark the k inputs directly. We checked Proposition 1 numerically by simulating U_{FQ} on every canonical input for $n = 4$ and $n = 6$, for all three families. The largest deviation from the stated action was 3×10^{-15}.

Proposition 3: Searching all 2^n inputs, with every x such that $f(x) \in \mathcal{M}$ marked, needs the same number of iterations and reaches the same success probability as searching X_0.

Proof: Since $f(x) = f(x \oplus s)$, marked inputs come in pairs, so the full space holds $2k$ of them. Hence $\sin^2 \theta' = 2k/2^n = k/2^{n-1} = \sin^2 \theta$, and Proposition 2 gives the same iteration count and success probability. ∎

The restriction to canonical representatives therefore saves no Grover iterations. Its benefit is structural: the search register loses one qubit, and every multi-controlled gate loses one control.

D. Cost Model and the Cost of Coherence

We decompose each multi-controlled X with $n_c \geq 3$ controls into a chain of $2n_c - 3$ Tooli gates that uses $n_c - 2$ work qubits [22]. The $2n_c - 4$ ladder Toolis come in compute and uncompute pairs, so we implement them as relative-phase Toolis with three CNOTs and four T gates [25]. Only the Tooli that flips the target stays a full Tooli, with six CNOTs and seven T gates [23], [24]. A multi-controlled X with n_c controls therefore uses $8n_c - 9$ T gates in this decomposition. The formula also holds for $n_c = 2$. The next result gives the cost of both oracles for any f.

Proposition 4 (Cost of coherence): Let $n \geq 4$, and let U_f use T_f T gates. Per iteration, the coherent oracle and diusion use $2T_f + |\mathcal{M}|(8n-17) + (8n-25)$ T gates, and the compile-time oracle with one gate per marked input and diusion use $(k+1)(8n-25)$. If f is two-to-one and every element of $\mathcal{M}$ has a preimage, then $k = |\mathcal{M}|$, and the coherent oracle needs exactly $2T_f + 8|\mathcal{M}|$ more T gates per iteration.

Proof: $P_{\mathcal{M}}$ holds $|\mathcal{M}|$ multi-controlled Z gates on y, each a multi-controlled X with $n-1$ controls and $8n-17$ T gates. Diusion holds one with $n-2$ controls and $8n-25$ T gates, and U_f runs twice. The compile-time oracle holds k gates with $n-2$ controls. Under the last two conditions, each element of $\mathcal{M}$ has exactly two preimages, which form one coset with one canonical member, so $k = |\mathcal{M}|$. Subtracting gives $2T_f + |\mathcal{M}|\,[(8n-17) - (8n-25)] = 2T_f + 8|\mathcal{M}|$. ∎

If g is given as another circuit with T_g T gates, T_g replaces $|\mathcal{M}|(8n-17)$ in both statements. Per iteration, the coherent oracle therefore needs fewer T gates exactly when $2T_f + T_g$ is below the T-count of the compile-time marking circuit, whichever synthesis produced it. The price of coherence is thus the two evaluations of f plus the dierence between testing outputs and listing inputs.

Threshold tests. A test can mark many outputs with few gates. For $\mathcal{M} = \{y : y < \tau\}$, $P_{\mathcal{M}}$ needs one multi-controlled Z per 1 bit of τ, on that bit and the bits above it. These gates mark disjoint sets whose union is $\mathcal{M}$, so Lemma 2 holds, and T_g does not grow with $|\mathcal{M}|$. A compiler can also compress the list, so we compare three compile-time syntheses: a cube cover, the cheapest fixed-polarity Reed–Muller form, and a cover by ane subspaces, an XOR-AND-XOR form [13] (Section S4). We add a synthetic family $\mathcal{R}$, the quadratic layer of family $\mathcal{N}$ followed by the linear layer of family $\mathcal{L}$, shaped like one round of a substitution–permutation cipher but not a real cipher, with $T_f = 7t$. Lemma 1 extends to it (Section S4), and Section IV-C reports the outcome.

For family $\mathcal{E}$, U_f holds $2c$ multi-controlled X gates with $n-1$ controls, so $T_f = 2c(8n-17)$. Over R iterations the Tooli and T counts are

$$\begin{aligned} N_{\mathrm{Tof}}^{\mathrm{FQ}} &= R\,[(4c + |\mathcal{M}|)(2n-5) + (2n-7)], \\ N_{T}^{\mathrm{FQ}} &= R\,[(4c + |\mathcal{M}|)(8n-17) + (8n-25)], \end{aligned} \tag{1}$$

and the compile-time oracle needs

$$N_{\mathrm{Tof}}^{\mathrm{CT}} = R\,(k+1)(2n-7), \qquad N_{T}^{\mathrm{CT}} = R\,(k+1)(8n-25). \tag{2}$$

Proposition 5: For family $\mathcal{E}$ with $\delta \notin \mathcal{M}$ and at least one marked input, $k \leq c$ and $N_{\mathrm{Tof}}^{\mathrm{FQ}}/N_{\mathrm{Tof}}^{\mathrm{CT}} > (4c + |\mathcal{M}| + 1)/(c+1) \geq 3$. The same bound holds for T gates.

Proof: If $\delta \notin \mathcal{M}$, only exceptional inputs can be marked. Each exceptional coset has one canonical member, so $k \leq c$. Since $2n-5 > 2n-7 > 0$ for $n \geq 4$, dividing (1) by (2) gives a ratio greater than $(4c + |\mathcal{M}| + 1)/(k+1) \geq (4c + |\mathcal{M}| + 1)/(c+1)$. A marked input needs $|\mathcal{M}| \geq 1$ and $c \geq k \geq 1$, so the bound is at least $(4c+2)/(c+1) \geq 3$. The T-gate case follows in the same way from $8n - 17 > 8n - 25 > 0$. ∎

A cube cover can only lower N^{CT}, so the bound holds against it as well. When $\delta \in \mathcal{M}$, all but at most c canonical inputs are marked, and a random guess succeeds with probability at least $1 - c/2^{n-1}$. In D3, the two evaluations of f take 76% of the T gates of an iteration at $n = 4$ and 73% at $n = 14$ (Table S2).

For family $\mathcal{L}$, $T_f = 0$, and f is two-to-one. When its image contains $\mathcal{M}$, Proposition 4 gives the coherent oracle only $8|\mathcal{M}|$ more T gates per iteration, and the T-count ratio $[|\mathcal{M}|(8n-17) + (8n-25)]/[(|\mathcal{M}|+1)(8n-25)]$ tends to one as n grows. Its extra cost moves to the CNOT network: two evaluations add twice the number of ones in L outside column b, up to $2n(n-1)$ CNOTs per iteration. For family $\mathcal{N}$, $T_f = 7t$, so in the same case the coherent oracle adds $14t + 8|\mathcal{M}|$ T gates per iteration; t moves the cost between that of family $\mathcal{L}$ at $t = 0$ and that of costly functions such as family $\mathcal{E}$.

Other decompositions. If a gate with n_c controls costs $T(n_c)$ T gates, the extra cost in Proposition 4 becomes $2T_f + |\mathcal{M}|\,[T(n-1) - T(n-2)]$, and Proposition 5 holds for every T that grows with n_c. Logical AND gates undone by measurement [26], $T(n_c) = 4(n_c - 1)$, also clear each flag for free, so U_f holds c gates and the bound falls to $(2c + |\mathcal{M}| + 1)/(c+1) \geq 2$. Measurement-based uncomputation [27] leaves one evaluation of f for families $\mathcal{L}$ and $\mathcal{N}$ (Section S3). Both measure in every iteration, which our hardware runs avoid.

Proposition 6 (Cost of listing): For family $\mathcal{E}$ with $\delta \notin \mathcal{M}$, the marked canonical inputs are the a_i with $o_i \in \mathcal{M}$, found with $O(cn)$ bit operations. For family $\mathcal{L}$, they are the vectors $L^{-1}m$ for $m \in \mathcal{M}$ whose bit b is 0, found with $O(n^3 + |\mathcal{M}|\,n^2)$ bit operations. For family $\mathcal{N}$, they are found with $O(|\mathcal{M}|(n+t))$ bit operations.

Proof: For family $\mathcal{E}$, inputs outside the exceptional cosets map to $\delta \notin \mathcal{M}$, so only the canonical members a_i can be marked. Checking the c outputs o_i against a hash set of $\mathcal{M}$ costs $O(cn)$. For family $\mathcal{L}$, the value $x \oplus x_b s$ always has bit b equal to 0, so $f(x) = m$ has a solution exactly when $L^{-1}m$ has bit b equal to 0, and that vector is the canonical solution. Inverting L by Gaussian elimination costs $O(n^3)$, and each product $L^{-1}m$ costs $O(n^2)$. For family $\mathcal{N}$, F is triangular, so $F(z) = m$ is solved bit by bit in increasing order. Each bit adds the products of bits already found, which costs $O(n+t)$ per element of $\mathcal{M}$, and the solution is kept if its bit b is 0. ∎

If f is available only as a black box, no such shortcut exists. Random sampling of canonical inputs then needs $(2^{n-1}+1)/(k+1)$ evaluations of f on average, the mean position of the first marked input in a random order. The

compile-time oracle can be built only after a classical search that already solves the problem.

The coherent oracle allocates $3n-1$ qubits for family $\mathcal{E}$ and $3n-2$ for families $\mathcal{L}$ and $\mathcal{N}$, which need no flag. The compile-time oracle needs n input qubits and $n-4$ work qubits for $n \geq 5$.

E. Cost of Finding the Period and of the Whole Pipeline

The period comes from Simon's algorithm. Each sample applies $H^{\otimes n}$, U_f on all inputs, and $H^{\otimes n}$ again, and returns a y with $y \cdot s = 0$, where $y \cdot s = \bigoplus_i y_i s_i$. Once the samples span the $(n-1)$-dimensional space orthogonal to s, the period is determined. This takes $O(n)$ samples when $f(x) = f(y)$ holds only for $y \in \{x, x \oplus s\}$ [19], as it does for families $\mathcal{L}$ and $\mathcal{N}$. Family $\mathcal{E}$ breaks that condition, since all inputs outside the exceptional cosets share one output.

Proposition 7: Let f be in family $\mathcal{E}$ with c exceptional cosets whose outputs are distinct and dier from δ. Every Simon sample $y \neq 0$ then has probability at most $4c(c+1)/4^n$, a sample is nonzero with probability less than $c(c+1)2^{1-n}$, and recovering s needs more than $(n-1)2^{n-1}/(c(c+1))$ samples in expectation.

Proof: With $N = 2^n$, a sample equals y with probability $\sum_z |N^{-1} \sum_{x:f(x)=z} (-1)^{x \cdot y}|^2$. If $y \cdot s = 1$, the two members of every coset cancel. Take $y \neq 0$ with $y \cdot s = 0$ and write $\chi_i = (-1)^{a_i \cdot y}$. Each exceptional output collects one coset, whose sum is $2\chi_i$, and contributes $4/N^2$. The default output collects all other inputs, and since the sum over all inputs is 0, its sum is $-2\sum_i \chi_i$, of magnitude at most $2c$. Hence y has probability at most $(4c + 4c^2)/N^2$, and since fewer than 2^{n-1} such y exist, a sample is nonzero with probability less than $c(c+1)2^{1-n}$. A zero sample carries no information, and spanning a space of dimension $n-1$ needs $n-1$ nonzero samples. ∎

The bound is exponential in n for any fixed number of exceptions, so the $O(n)$ bound of Kaplan et al. [1] does not apply. In their terms, the collision probability $\varepsilon = \max_{u \notin \{0,s\}} \Pr_x[f(x \oplus u) = f(x)]$ is 0.7500 at $n = 4$ and 0.9998 at $n = 14$ for our family $\mathcal{E}$ benchmark, and zero for families $\mathcal{L}$ and $\mathcal{N}$. In Table S16, the mean number of samples for family $\mathcal{E}$ grows from 11 at $n = 4$ to 30 368 at $n = 14$, above the bound of 17 749. For families $\mathcal{L}$ and $\mathcal{N}$ the samples are uniform on the space orthogonal to s, and fewer than $n+1$ suce on average.

The whole pipeline. A coherent search that must first find the period costs, in T gates,

$$T_{\text{pipe}} = S\,T_S + R\,T_{\text{it}}, \tag{3}$$

where S is the expected number of Simon samples, T_S the T-count of one sample, and T_{it} that of one search iteration. A compile-time search pays instead the classical listing cost of Proposition 6 and $R\,T_{\text{it}}^{\text{CT}}$, with $T_{\text{it}}^{\text{CT}}$ the T-count of one of its iterations. For family $\mathcal{E}$ the Simon stage costs 13 to 483 times the search (Table S16), so the period-first pipeline is not worth running. For family $\mathcal{N}$ the Simon stage costs at most 38% of the total, and for family $\mathcal{L}$ it needs no T gates. In the experiments we take s from the function description, as a compile-time design would.

Algorithm 1: Oracle selection pass.

Input: f as a circuit U_f or a structure; marked set $\mathcal{M}$; classical budget B; target: error-corrected, or a device with p_{eff}.
Output: An oracle circuit and a cost report.

```
Estimate the listing cost C_list (Proposition 6)
D ← {coherent}; if C_list ≤ B then
  D ← D ∪ {compile-time}
if the target is error-corrected then
    return the design in D with fewer T gates, breaking ties by qubits
else
    foreach design in D and each R' ≤ R do
        Route the circuit and count its two-qubit gates G
        Predict P with Eq. (4)
    return the pair with the highest P, and warn if P ≤ k/2^{n-1}
```

F. Oracle Selection Pass

We turn the comparison into a compiler pass (Algorithm 1). Its inputs are a period-restricted search problem, a classical budget B, and a target. It first estimates the cost of listing the marked inputs from the function description (Proposition 6); a black box allows only random sampling. If listing fits the budget, both designs are candidates, and otherwise only the coherent oracle is. The pass selects among the representations it is given; a stronger synthesis enters it as one more candidate. For an error-corrected target the pass costs each candidate with Proposition 4 and keeps the one with fewer T gates. For a current device it routes every candidate for every iteration count $R' \leq R$ and predicts its success with a calibrated heuristic model of the backend.

$$P = \Phi\, P_0 + (1 - \Phi)\, k/2^n, \qquad \Phi = (1 - p_{\text{eff}})^G. \tag{4}$$

In Eq. (4), P_0 is the noiseless success, G the routed two-qubit count, p_{eff} one error rate per routed two-qubit gate, fitted to a single calibration circuit on the target, and Φ the probability that no error occurs. The model lumps every error source into p_{eff}, in the spirit of the digital error model of [44], and a failure sends the register to the chance level $k/2^n$, that of a fully mixed register. We use it as a ranking heuristic: the pass needs the order of the candidates, not their exact success. The pass returns the design and R' with the highest prediction and warns when that prediction does not exceed the no-search level $k/2^{n-1}$, the success of measuring the prepared state without any search. As a Qiskit transpiler pass it takes at most 0.07 s for an error-corrected target up to $n = 14$, and up to 18 times one routing of its output for a device (Section S9).

IV. Experiments

This section describes the setup, then reports the cost of the oracles and how it varies with the function, the ablation of the compilation steps, the comparison under noise, the hardware runs, and the validation of the selection pass.

A. Setup

Metrics. We measure three kinds of quantity. Logical cost, in Tooli gates, T gates, and T-depth, follows from the circuit

alone and dominates time and space on an error-corrected machine, where every T gate consumes a costly resource state [24], [32]. Compiled cost counts two-qubit gates and depth after transpilation, the compilation of a circuit to the gates and connectivity of a device. It predicts how much noise a circuit collects on current hardware: on the Heron r2 model the median two-qubit error is 3.9×10^{-3}, about 17 times the single-qubit error. Success is the probability that the measured x is a marked canonical input. Measuring the prepared state without any search gives $k/2^{n-1}$, which is 0.250 at $n = 4$ and 0.063 at $n = 6$; we call this the no-search level and success above it amplification. A fully mixed register gives $k/2^n$, which is 0.125 and 0.031; we call this the chance level.

Benchmark families. For even n we set $s = (10)^{n/2}$, so $b = 0$. For family $\mathcal{E}$, $\delta = 1^n$, $\mathcal{M} = \{0^n, 0^{n-2}11\}$, and the exceptional cosets are $\{0^n, s\} \mapsto 0^n$ and $\{0^{n-2}11,\ 0^{n-2}11 \oplus s\} \mapsto 0^{n-2}11$, so $c = 2$ and $|\mathcal{M}| = k = 2$. For family $\mathcal{L}$ we draw L at random with a fixed seed per n. For family $\mathcal{N}$ we draw $t = n$ products with a fixed seed, capped by the number admissible, so $t = 1$ at $n = 4$. In both we mark the images of the same two canonical inputs, so $k = 2$ and the compile-time oracle is the same for all three families. The families were fixed in advance and not tuned.

Designs. We compare the four variants of family $\mathcal{E}$ (D0 to D3), and the coherent oracles for families $\mathcal{L}$ (Lin) and $\mathcal{N}$ (Quad), with the compile-time oracle (CT). All are built by the same code, with the same decompositions and the same R. CT is the cube cover of Section III-B, which for these instances places one gate per marked input.

Simulation. We use Qiskit 1.4.4 [33] and its simulator Qiskit Aer 0.17.1. We transpile to the gate set $\{cx, id, rz, sx, x\}$: the CNOT (cx), the identity (id), rotations about the Z axis (rz), the square root of X (sx), and the Pauli X gate (x). We use Qiskit's optimization level 1 (of 0 to 3) with all-to-all connectivity, so any two qubits may interact. The noise model applies a depolarizing channel of strength p after every one- and two-qubit gate: with probability p it replaces the state of the qubits the gate acts on by the fully mixed state. It includes rz, which IBM hardware applies without error by adjusting later pulses, so the model is pessimistic, and it leaves measurement noiseless. At $n = 4$ we compute exact probabilities from the density matrix. At $n = 6$ the density matrices of the coherent oracles are too large, and we sample 4 000 noisy trajectories per point, Monte Carlo runs with random errors, and report 95% Wilson intervals for the resulting proportions.

Routing. We transpile to the coupling map, the graph of qubit pairs that can interact, and the native gates $\{cz, id, rz, sx, x\}$ of the 156-qubit Heron r2 model FakeFez from qiskit-ibm-runtime 0.40.1, where cz is the controlled-Z gate. We use optimization level 3 and 10 random seeds of the transpiler at $n = 4$ and 5 at $n = 6$, 8, and 10. We compare against the same optimization level and gate set without a coupling map.

Hardware. We ran through IBM's SamplerV2 interface, which returns measurement counts, with 8 192 shots, or repetitions, per circuit and run. On 3 June 2026 (11:23 to 11:25 UTC) D3 ran alone on `ibm_fez`, in five jobs per size. On 11 October 2026 five jobs each held all 14 circuits of Table III, so the designs of a job share one calibration; they ran on `ibm_fez` (02:55 to 03:24 UTC) and on `ibm_marrakesh` (23:56 to 23:59 UTC). We transpiled at optimization level 3 against the calibration of the day, the error rates and coherence times that IBM measures for each qubit and coupler. We used no mid-circuit measurement and no error mitigation, the post-processing that corrects results for noise. Section S7 lists every job, its success, and the calibration in force. We report the mean over runs, the sample standard deviation, and a 95% interval with the Student-t quantile 2.776 for four degrees of freedom.

Device models. To test the selection pass beyond one device, we use the noise models of five IBM devices from qiskit-ibm-runtime, built from their published calibrations. They span three processor generations: FakeFez and FakeMarrakesh (Heron r2), FakeTorino (Heron r1), and FakeSherbrooke and FakeBrisbane (Eagle r3). On each we route every candidate with transpiler seed 0, and with seeds 1 and 2 to test robustness, and simulate it with the device's noise model, with 2 000 shots, or 500 for circuits wider than 16 qubits. The budget B admits listing from the function description but not black-box enumeration, so each decision is made with and without listing.

B. Cost of the Oracles and How It Varies

Table II gives the cost of the coherent oracles relative to CT from $n = 4$ to $n = 14$, and Table S1 and Fig. S1 give the absolute counts. For family $\mathcal{E}$, D3 needs 3.97 to 7.48 times the T gates of CT, in line with Proposition 5. For family $\mathcal{L}$ the ratio falls from 1.76 at $n = 4$ to 1.06 at $n = 14$. For family $\mathcal{N}$ it lies between the two, at 1.81 at $n = 14$, as Proposition 4 predicts with $T_f = 7t$.

The two-qubit counts tell a dierent story. After transpilation, D3 needs 2.8 to 6.0 times the two-qubit gates of CT. The CNOT network of family $\mathcal{L}$ grows with n^2, so Lin needs 2.0 to 2.9 times as many as CT, and at $n = 14$ it needs more than D3. The cost of coherence therefore depends on the metric. The T-count, which dominates error-corrected cost, favors compact circuits, while the two-qubit count, which limits current hardware, penalizes dense CNOT networks. Fig. S2 shows how the price of coherence moves with the number of exceptions, products, and ones in L.

The T-depth, which sets the running time of an error-corrected machine when many T gates can run at once, ranks the designs as the T-count does at every n (Table S1). D3 needs 3.97 to 10.25 times the T-depth of CT.

C. Threshold Tests and the Choice by Target

Fig. 2 compares the coherent oracle with the cheapest of the three compile-time syntheses of Section III-D when a threshold test describes $\mathcal{M}$. It covers 20 random instances per family, $n = 4$ to 14, and every threshold with $\tau \leq 2^{n-1}/4$ (instance 0 in Tables S4 and S5).

For families $\mathcal{N}$ and $\mathcal{L}$ some synthesis needs no more T gates than the coherent oracle in all 400 and all 400 cases, since their marked inputs form a few cubes and one ane subspace.

TABLE II
COST OF THE COHERENT ORACLES (D3, LIN, QUAD FOR FAMILIES $\mathcal{E}$, $\mathcal{L}$, $\mathcal{N}$) RELATIVE TO THE COMPILE-TIME ORACLE (CT) FOR THE WHOLE SEARCH. T-COUNT: PROPOSITION 4, CONFIRMED ON EVERY EXPANDED CIRCUIT. TWO-QUBIT: TRANSPILED AT OPTIMIZATION LEVEL 1, ALL-TO-ALL. ABSOLUTE COUNTS: TABLE S1.

	T-count ratio			Two-qubit ratio			CT	
n	D3	Lin	Quad	D3	Lin	Quad	T	Two-qubit
4	7.48	1.76	2.43	6.0	2.0	2.3	21	18
6	4.83	1.23	2.45	3.8	2.4	3.1	207	126
8	4.35	1.14	2.09	3.3	2.3	2.8	702	396
10	4.15	1.10	1.95	3.0	2.5	2.6	1 980	1 080
12	4.04	1.08	1.86	2.9	2.8	2.5	5 325	2 850
14	3.97	1.06	1.81	2.8	2.9	2.4	13 050	6 900

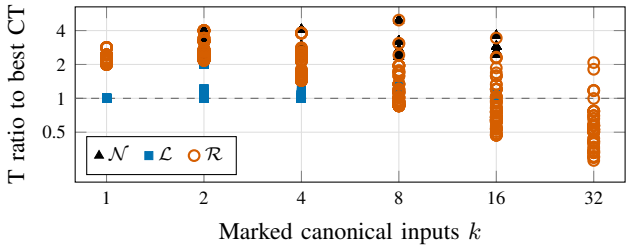


Fig. 2. T gates per iteration of the coherent oracle over those of the cheapest compile-time synthesis (cube, Reed–Muller, or ane cover), for threshold tests $\mathcal{M} = \{y : y < \tau\}$, against the number k of marked canonical inputs. All 20 instances per family, $n = 4$ to 14, and $\tau \le 2^{n-1}/4$: 400, 400, and 540 cases. Below the dashed line the coherent oracle needs fewer T gates. Logarithmic axes; coinciding points overlap.

In family $\mathcal{R}$ the two layers scatter the marked inputs. The coherent oracle then needs fewer T gates in 160 of the 233 cases with $k \ge 8$, a share that rises with n to 59 of 63 at $n = 14$, and in none with $k < 8$. At $n = 14$ and $k = 32$ it needs 330 against 655 T gates per iteration. The selection pass for an error-corrected target, given all three syntheses, picks it in exactly these cases.

A current device reverses the choice. The CNOT networks for L give the coherent oracle more two-qubit gates than the best compile-time synthesis in all 27 cases of instance 0. We tested one case in which the error-corrected pass picks the coherent oracle, $n = 8$ and $k = 16$, on the five device models. There it needs 158 against 209 T gates per iteration, but after routing at $R' = 1$ it needs 910 to 951 two-qubit gates against 398 to 423 for CT. The device branch picks CT at $R' = 1$ on every device. Noisy simulation gives CT 0.141 to 0.352 and the coherent oracle 0.100 to 0.148, so CT is the best simulated candidate on every device; the coherent oracle at $R' = 2$, predicted at the chance level, was not simulated (Table S13). The pass thus emits dierent oracles for one function on the two targets.

D. Ablation of the Compilation Steps

Fig. 3(a) and Table S17 show how each step changes the cost and the noisy success at $n = 4$. Together the four steps cut the two-qubit count by a factor of 5.8 at $n = 4$ and 7.2 at $n = 6$. At $n = 4$ the canonical restriction (D0 to D1) gives the largest factor, 2.2. At $n = 6$ the relative-phase Toolis

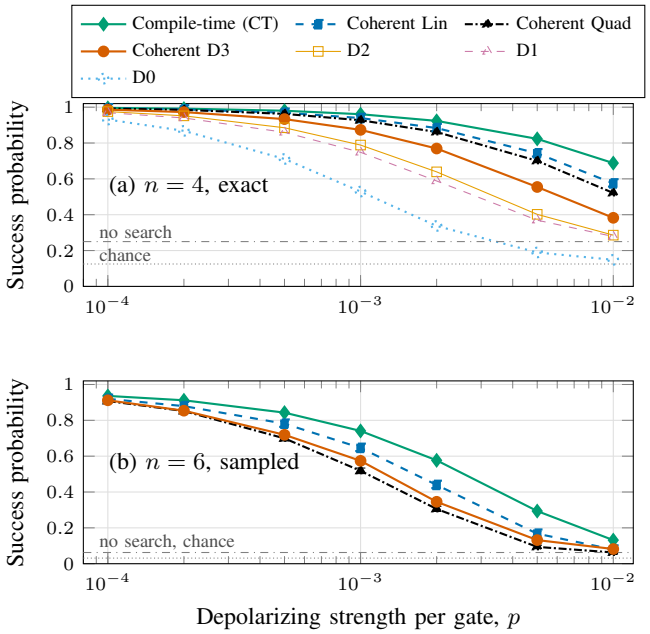


Fig. 3. Success probability against the per-gate depolarizing strength. (a) $n = 4$: design variants D0 to D3, the coherent oracles for families $\mathcal{L}$ (Lin) and $\mathcal{N}$ (Quad), and the compile-time oracle, from exact density matrices. (b) $n = 6$: CT exact; D3, Lin, and Quad from 4 000 sampled trajectories per point, with 95% Wilson intervals. Reference lines: no-search level $k/2^{n-1}$ and chance level $k/2^n$.

(D2 to D3) give the largest, 2.9. Each step raises success at every noise level we simulated. At $p = 10^{-3}$ success rises from 0.529 for D0 to 0.873 for D3.

E. Coherent Versus Compile-Time Under Noise

The compile-time oracle leads every coherent oracle at every simulated noise level (Fig. 3). At $n = 4$ Lin also leads D3 at every level, and Quad, with its single product, lies between CT and D3. The lead of CT over D3 is $+0.010$ at $p = 10^{-4}$, $+0.087$ at 10^{-3}, and $+0.305$ at 10^{-2}. At $n = 6$ and $p = 10^{-3}$, CT reaches 0.741, Lin 0.646 (95% interval 0.631 to 0.661), and D3 0.574 (0.559 to 0.589). The model of Eq. (4), with p in place of p_{eff}, captures this ordering (Section S10).

F. Hardware Results and Design Comparison

D3, the most costly design, first ran alone. On 3 June 2026 its success at $n = 4$ is 0.368 (95% interval 0.326 to 0.410), above the no-search level of 0.250 in every run (one-sided t-test, p-value 7×10^{-4}). At $n = 6$ the mean is 0.033 (0.030 to 0.036), which contains the chance level of 0.031 and lies below the no-search level of 0.063. A noise model built from the calibration of that day explains only 55% of the eective error (Section S7-A).

Table III gives the run of every design, with matched routing, on 11 October 2026 on both processors, beside two predictions for the same routed circuits: the FakeFez noise model and Eq. (4) with the June rate (Fig. S3). On both, every design amplifies at $n = 4$, ranked as both predictions do: CT, Lin, Quad, and D3, the order of their routed two-qubit counts. At $n = 6$ CT at $R = 2$ beats CT at the noiseless optimum $R = 3$

TABLE III
DESIGNS ON THE HERON R2 MODEL AND ON TWO HERON R2 PROCESSORS. G: ROUTED TWO-QUBIT COUNT (FAKEFEZ, SEED 0). FAKEFEZ: SUCCESS UNDER THE NOISE MODEL OF THE CALIBRATION SNAPSHOT, WITH 95% WILSON INTERVAL. PRED.: EQ. (4) WITH THE RATE FITTED TO D3 AT $n = 4$ ON `ibm_fez` ON 3 JUNE 2026. HARDWARE: MEAN OF FIVE RUNS ON 11 OCTOBER 2026 ON `ibm_fez` (FEZ) AND `ibm_marrakesh` (MARR.), WITH THE SAME G. NO-SEARCH LEVEL 0.250 AND 0.063; CHANCE LEVEL 0.125 AND 0.031.

						Hardware	
Design	n	R	G	FakeFez [95%]	Pred.	Fez	Marr.
No search	4	0	0	0.253 [0.247, 0.259]	0.250	0.259	0.252
D3	4	1	211	0.623 [0.602, 0.644]	0.374	0.460	0.371
Quad	4	1	82	0.770 [0.751, 0.788]	0.662	0.640	0.571
Lin	4	1	64	0.804 [0.786, 0.820]	0.723	0.652	0.585
CT	4	1	27	0.895 [0.881, 0.908]	0.870	0.883	0.890
No search	6	0	0	0.063 [0.060, 0.067]	0.063	0.066	0.059
D3	6	3	1 548	0.118 [0.105, 0.133]	0.031	0.029	0.026
	6	1	453	0.152 [0.137, 0.168]	0.061	0.049	0.061
Quad	6	3	1 160	0.093 [0.081, 0.107]	0.032	0.081	0.070
	6	1	325	0.178 [0.162, 0.195]	0.095	0.077	0.088
Lin	6	3	1 058	0.109 [0.096, 0.123]	0.033	0.092	0.087
	6	1	329	0.181 [0.164, 0.198]	0.093	0.082	0.083
CT	6	3	186	0.477 [0.455, 0.498]	0.338	0.285	0.136
	6	2	124	0.550 [0.528, 0.571]	0.450	0.353	0.187

on both, as both predictions expect. Lin and Quad also stay above the no-search level at $n = 6$ on both processors, while D3 does not at either R; the measured no-search circuits agree with these levels, and the comparisons hold against them too. Absolute success varies: CT at $n = 6$ and $R = 2$ reaches 0.353 on `ibm_fez` but 0.187 on `ibm_marrakesh`, and D3 at $n = 4$ reached 0.460 on `ibm_fez`, outside its June interval. On `ibm_fez` the snapshot model overrates all 11 other search circuits, by 0.100 on average, while Eq. (4) errs by 0.037 and ranks them with a rank correlation of 0.88.

G. Validation of the Selection Pass

We test Eq. (4), on which the device branch of the pass rests, on data it was not fitted to. On five device models (Table IV), the rate fitted to D3 at $n = 4$ predicts the other 15 candidates with a mean absolute error of 0.060. The model overrates every compile-time circuit and underrates the deepest coherent ones, so its errors reach 0.252, but its ranking holds, with a rank correlation of at least 0.87 on every device. The pass picks the best simulated candidate in 29 of 30 decisions: CT at $R = 2$ with listing and the coherent oracle at $R = 1$ without (Table S11). Its one miss loses 0.027 in success, within twice its sampling error. The decisions fall into six types, three families with and without listing, each repeated on five devices, so they are not independent tests.

The choices are robust: on every device each holds for rates from 0.71 to 2.09 times the fitted one, and on FakeFez up to 4.1 times, which covers the rate fitted on hardware in June, 2.2 times the FakeFez rate. Routed with transpiler seeds 1 and 2, and with the rate refitted for each seed, the pass keeps all 30 choices. Over the 90 decisions, the same types on three seeds, it picks the best candidate in 83, and its seven misses are coherent oracles without listing, each within 2.3 sampling errors of the best (Table S15). With the rate fitted

TABLE IV
VALIDATION OF THE SELECTION PASS ON FIVE DEVICE MODELS. p_{eff}: RATE FITTED TO D3 AT $n = 4$. MAE: MEAN ABSOLUTE ERROR OF THE PREDICTED SUCCESS OF THE OTHER 15 CANDIDATES AGAINST SIMULATION WITH THE DEVICE'S NOISE MODEL. ρ: SPEARMAN RANK CORRELATION. CHOICES: DECISIONS IN WHICH THE PASS PICKS THE BEST SIMULATED CANDIDATE, OUT OF SIX AT $n = 6$ (THREE FAMILIES, WITH AND WITHOUT LISTING). REGRET: LARGEST LOSS IN SUCCESS. PER CANDIDATE: TABLE S12.

Device	Processor	p_{eff}	MAE	ρ	Choices	Regret
FakeFez	Heron r2	2.7×10^{-3}	0.050	0.96	6/6	0.000
FakeMarrakesh	Heron r2	2.7×10^{-3}	0.051	0.94	5/6	0.027
FakeTorino	Heron r1	4.5×10^{-3}	0.072	0.92	6/6	0.000
FakeSherbrooke	Eagle r3	5.7×10^{-3}	0.062	0.87	6/6	0.000
FakeBrisbane	Eagle r3	4.8×10^{-3}	0.064	0.88	6/6	0.000

on another device it picks the best in 332 of 360. Removing one component at a time costs matches (Table S14): without the noise model the pass matches 2 of the 90, without the amplitude term 38, losing up to 0.203, and without routing 77. Without the listing check it picks CT in all 45 black-box decisions, where CT cannot be built. At $n = 8$, where we simulate CT alone for the benchmark (Section S10), the pass picks the best R on three of the five devices, and neither of the first two variants does on any.

On hardware, the pass picks the best measured circuit in four of the six decisions at $n = 6$ on `ibm_fez` and five on `ibm_marrakesh`, among the circuits run: CT at $R = 2$ with listing in all three families on both. Without listing, Lin did better at $R = 3$ than at the chosen $R = 1$ on both processors (by 3.3 and 3.0 sampling errors), and Quad on `ibm_fez` only, by 1.9. Eq. (4) predicts these deeper circuits at the chance level, yet they measured above the no-search level.

V. DISCUSSION

This section states when each oracle design fits and where the choice arises in practice, draws the implications for compilers and benchmarks, and lists the limits of this study.

A. When Is a Coherent Oracle Worth Its Cost?

The price of coherence (Proposition 4) is high for family $\mathcal{E}$, low in T gates but high in two-qubit gates for family $\mathcal{L}$, and linear in the number of Tooli gates for family $\mathcal{N}$. A cheap test can oset it only when no synthesis compresses the listed inputs, as for our cipher-like round (Section IV-C). Even then, the routed two-qubit count of a current device can reverse the choice.

Two results screen a workload before a period-first pipeline runs. The restriction saves one qubit and one control per gate but no iterations (Proposition 3). Finding the period dominates the pipeline when f has many collisions besides those forced by s (Proposition 7, Eq. (3), Table S16). These results suggest that the pipeline is worth running only when f satisfies Simon's condition and the marked inputs cannot be listed.

Our results do not show a quantum advantage: they assume that a search is run and ask which oracle to compile. An advantage needs marked inputs that no classical program can

list cheaply, as in key search, and then only the coherent oracle exists. Its price is two evaluations of f per iteration (Proposition 4), against one evaluation per guess for classical sampling. The quadratic saving in queries survives this constant factor, since search cost grows with the square root of the search space and sampling cost linearly. The factor still counts: a factor γ in the cost per query multiplies by γ^2 the size at which search starts to beat sampling.

B. Where the Choice Arises

The choice between the two oracles arises in at least four kinds of workload. In key search, no classical program can list the secret keys without solving the problem, so only a coherent oracle can be built [2], [16]. In a search over computed values, such as Imre's search for the extreme value in an unsorted database by a sequence of thresholds [45], the threshold test is a membership test g (Section III-B), which the coherent oracle takes directly. Quantum counting [43], quantum existence testing [45], and search without knowing k [20] all need a coherent oracle, since a list of the marked inputs would already give k.

C. Implications for Compilers and Benchmarks

The selection pass weighs three costs: classical construction, logical quantum cost, and execution cost on the target. Existing tools supply the parts of the selection pass: synthesis of U_f [12], [13], automatic uncomputation [30], table lookups [32], and routing [34], [35]. What they lack is the comparison between the two designs, which Proposition 4 supplies in closed form.

The same comparison matters for benchmarks, which usually build the oracle from a chosen marked state [7], [8], [42]. For family $\mathcal{E}$ at $n = 4$, such an oracle needs 18 two-qubit gates against 108 for the coherent oracle that a real search requires, and on `ibm_fez` it reached 0.883 against 0.460. A benchmark result therefore depends on which oracle it uses, and reports should state it or give both [10], [11].

D. Limitations and Threats to Validity

Construct validity. Our success metric counts only marked canonical inputs and ignores their equally valid partners, which would raise every success value and reference level together. Our cost metrics depend on the decompositions of Section III-D, which also bounds the eect of other decompositions. We do not compare numerically with published HQDOA values, because its marking rule marks no input under the period condition (Section II-E).

Internal validity. All designs share one code base, one transpiler version, and fixed seeds, so in simulation their dierences come from the oracle alone. On hardware the transpiler placed each circuit on its own qubits, so dierences there mix the oracle with qubit quality (Table S9). The routed counts vary with the transpiler seed, and we report their range (Table S10).

External validity. The three families are small and structured, and their marked inputs can be listed classically, so they measure oracle cost, not advantage (Section V-A; Section S5 compares query counts with classical sampling). They span costly, linear, and quadratic functions and, in the threshold study, a cipher-like round, with 20 random instances per family, but not arithmetic circuits or full ciphers. For family $\mathcal{R}$ the outcome depends on the instance at every size from $n = 8$ to 14. Beyond $n = 6$ we report cost and routing. At $n = 8$ noise is simulated only for CT on the benchmark (Section IV-G) and for one family $\mathcal{R}$ case, including its coherent oracle at $R' = 1$ (Section IV-C); nothing beyond $n = 6$ runs on hardware.

Noise models. The depolarizing and calibration models omit coherent errors, crosstalk, and leakage. Eq. (4) is a heuristic with one parameter per device. It ranks designs well in our tests (Section IV-G) but does not predict hardware success in general. Per-gate models that multiply calibrated fidelities [44] could capture variation between qubits, though the model built from our calibration overrated success (Section IV-F). Algorithm 1 can use any such predictor.

Hardware. Hardware covers two processors of one type, two dates, five runs per circuit, and no mitigation. Success moved between dates and between processors by more than its intervals, so a comparison on one processor and date carries variation that its intervals do not show. The hardware tests the pass in six decisions at $n = 6$ per processor, among the iteration counts we ran.

VI. Conclusion

We compared coherent and compile-time oracles for period-restricted Grover search. We proved the coherent oracle correct and derived exact costs for a stated gate decomposition: without mid-circuit measurement, coherence costs two evaluations of f per iteration. In T gates this makes the coherent oracle more than three times as costly for exception-described functions with an unmarked default, and more than twice with logical AND gates undone by measurement. For linear circuits the extra T gates are nearly nil. A threshold test makes the coherent oracle cheaper only when no synthesis we tried compresses the listed inputs, as for our synthetic cipher-like round. For exception-described functions, finding the period also costs more than the search itself.

In simulation with the noise models of five IBM processors, the pass chose the best candidate in 83 of 90 decisions. On `ibm_fez` and `ibm_marrakesh` every design amplified at $n = 4$, and at $n = 6$ so did the list and the coherent oracles of families $\mathcal{L}$ and $\mathcal{N}$, but not D3. Oracle representation is therefore a compilation decision. A compiler should compare the two costs and set the iteration count with a calibrated device model, as our pass does, though on hardware our model underrated deep coherent circuits. It should emit a coherent oracle when the inputs cannot be listed, or when a cheap test marks inputs that no synthesis compresses. Benchmarks should state which oracle they use.

Two extensions follow. Searches that share one compiled and routed U_f, such as the threshold sequence of extreme-value search (Section V-B), call for selecting oracles jointly across the workload, with shared costs counted once. A mixed oracle that lists some marked inputs and tests the rest would generalize both designs.

REFERENCES

[1] M. Kaplan, G. Leurent, A. Leverrier, and M. Naya-Plasencia, "Breaking symmetric cryptosystems using quantum period finding," in *Advances in Cryptology – CRYPTO 2016*, ser. Lecture Notes in Computer Science. Berlin, Germany: Springer, 2016, pp. 207–237, doi: 10.1007/978-3-662-53008-5_8.

[2] G. Leander and A. May, "Grover meets Simon: Quantumly attacking the FX-construction," in *Advances in Cryptology – ASIACRYPT 2017*, ser. Lecture Notes in Computer Science. Cham, Switzerland: Springer, 2017, pp. 161–178, doi: 10.1007/978-3-319-70697-9_6.

[3] X. Bonnetain, A. Hosoyamada, M. Naya-Plasencia, Y. Sasaki, and A. Schrottenloher, "Quantum attacks without superposition queries: The oine Simon's algorithm," in *Advances in Cryptology – ASIACRYPT 2019*, ser. Lecture Notes in Computer Science. Cham, Switzerland: Springer, 2019, pp. 552–583, doi: 10.1007/978-3-030-34578-5_20.

[4] X. Bonnetain and S. Jaques, "Quantum period finding against symmetric primitives in practice," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2022, no. 1, pp. 1–27, Nov. 2021, doi: 10.46586/tches.v2022.i1.1-27.

[5] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*, 1996, pp. 212–219, doi: 10.1145/237814.237866.

[6] P. P. Patil, J. Tembhurne, and S. J. Choudhury, "Quantum state measurement system with dynamic oracle adaptation for enhanced parameter estimation in NISQ devices," *IEEE Trans. Instrum. Meas.*, vol. 75, pp. 1–8, 2026, doi: 10.1109/TIM.2026.3659640.

[7] C. Figgatt, D. Maslov, K. A. Landsman, N. M. Linke, S. Debnath, and C. Monroe, "Complete 3-qubit Grover search on a programmable quantum computer," *Nature Commun.*, vol. 8, p. 1918, Dec. 2017, doi: 10.1038/s41467-017-01904-7.

[8] M. AbuGhanem, "Characterizing Grover search algorithm on large-scale superconducting quantum computers," *Sci. Rep.*, vol. 15, no. 1, p. 1281, Jan. 2025, doi: 10.1038/s41598-024-80188-6.

[9] C. H. Bennett, "Logical reversibility of computation," *IBM J. Res. Develop.*, vol. 17, no. 6, pp. 525–532, Nov. 1973, doi: 10.1147/rd.176.0525.

[10] F. Leymann and J. Barzen, "The bitter truth about gate-based quantum algorithms in the NISQ era," *Quantum Sci. Technol.*, vol. 5, no. 4, p. 044007, Oct. 2020, doi: 10.1088/2058-9565/abae7d.

[11] N. Johansson and J.-Å. Larsson, "Quantum simulation logic, oracles, and the quantum advantage," *Entropy*, vol. 21, no. 8, p. 800, Aug. 2019, doi: 10.3390/e21080800.

[12] M. Soeken, M. Roetteler, N. Wiebe, and G. De Micheli, "LUT-based hierarchical reversible logic synthesis," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 9, pp. 1675–1688, Sep. 2019, doi: 10.1109/TCAD.2018.2859251.

[13] A. Bernasconi, A. Berti, V. Ciriani, G. M. Del Corso, and I. Fulginiti, "XOR-AND-XOR logic forms for autosymmetric functions and applications to quantum computing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 6, pp. 1861–1872, Jun. 2023, doi: 10.1109/TCAD.2022.3213214.

[14] R. Seidel, C. K.-U. Becker, S. Bock, N. Tcholtchev, I.-D. Gheorghe-Pop, and M. Hauswirth, "Automatic generation of Grover quantum oracles for arbitrary data structures," *Quantum Sci. Technol.*, vol. 8, no. 2, p. 025003, 2023, doi: 10.1088/2058-9565/acaf9d.

[15] L. Li, F. Voichick, K. Hietala, Y. Peng, X. Wu, and M. Hicks, "Verified compilation of quantum oracles," *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA2, pp. 589–615, 2022, doi: 10.1145/3563309.

[16] S. Jaques, M. Naehrig, M. Roetteler, and F. Virdia, "Implementing Grover oracles for quantum key search on AES and LowMC," in *Advances in Cryptology – EUROCRYPT 2020*, ser. Lecture Notes in Computer Science. Cham, Switzerland: Springer, 2020, pp. 280–310, revised version with corrected AES estimates: IACR Cryptology ePrint Archive, Report 2019/1146, Jun. 2023. doi: 10.1007/978-3-030-45724-2_10.

[17] D. Chung, S. Lee, D. Choi, and J. Lee, "Alternative tower field construction for quantum implementation of the AES S-box," *IEEE Trans. Comput.*, vol. 71, no. 10, pp. 2553–2564, Oct. 2022, doi: 10.1109/TC.2021.3135759.

[18] M. Zhang, T. Shi, W. Wu, and H. Sui, "Optimized quantum circuit of AES with interlacing-uncompute structure," *IEEE Trans. Comput.*, vol. 73, no. 11, pp. 2563–2575, Nov. 2024, doi: 10.1109/TC.2024.3449094.

[19] D. R. Simon, "On the power of quantum computation," in *Proc. 35th Annu. Symp. Found. Comput. Sci. (FOCS)*, 1994, pp. 116–123, doi: 10.1109/SFCS.1994.365701.

[20] M. Boyer, G. Brassard, P. Høyer, and A. Tapp, "Tight bounds on quantum searching," *Fortschr. Phys.*, vol. 46, no. 4–5, pp. 493–505, Jun. 1998, doi: 10.1002/(SICI)1521-3978(199806)46:4/5<493::AID-PROP493>3.0.CO;2-P.

[21] Q. Xia, Q. Zhu, H. Xie, and L. Yang, "Tight analysis of Grover-meets-Simon and Alg-PolyQ2 attacks via formalizing quantum rank-solving under deferred measurement," *Quantum Inf. Comput.*, vol. 25, no. 5, pp. 421–437, Dec. 2025, doi: 10.2478/qic-2025-0024.

[22] A. Barenco *et al.*, "Elementary gates for quantum computation," *Phys. Rev. A*, vol. 52, no. 5, pp. 3457–3467, Nov. 1995, doi: 10.1103/PhysRevA.52.3457.

[23] V. V. Shende and I. L. Markov, "On the CNOT-cost of TOFFOLI gates," *Quantum Inf. Comput.*, vol. 9, no. 5–6, pp. 461–486, May 2009, doi: 10.26421/qic8.5-6-8.

[24] M. Amy, D. Maslov, M. Mosca, and M. Roetteler, "A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 6, pp. 818–830, Jun. 2013, doi: 10.1109/TCAD.2013.2244643.

[25] D. Maslov, "Advantages of using relative-phase Tooli gates with an application to multiple control Tooli optimization," *Phys. Rev. A*, vol. 93, no. 2, p. 022311, Feb. 2016, doi: 10.1103/PhysRevA.93.022311.

[26] C. Gidney, "Halving the cost of quantum addition," *Quantum*, vol. 2, p. 74, Jun. 2018, doi: 10.22331/q-2018-06-18-74.

[27] A. Luongo, A. M. Miti, V. Narasimhachar, and A. Sireesh, "Measurement-based uncomputation of quantum circuits for modular arithmetic," in *Proc. 62nd ACM/IEEE Design Autom. Conf. (DAC)*, 2025, pp. 1–7, doi: 10.1109/DAC63849.2025.11132981.

[28] S. Dutta, S. Wang, A. Baksi, A. Chattopadhyay, and S. Maitra, "Exact space-depth trade-os in multicontrolled Tooli decomposition," *Phys. Rev. A*, vol. 111, no. 5, p. 052611, May 2025, doi: 10.1103/PhysRevA.111.052611.

[29] G. Li, Y. Gan, Z. Wu, X. Zhang, Z. Zhang, and J. Liu, "Stab-QRAM: A Cliord-only quantum oracle for ane Boolean data," arXiv:2509.26494 [quant-ph], preprint, 2025.

[30] A. Paradis, B. Bichsel, S. Steen, and M. Vechev, "Unqomp: Synthesizing uncomputation in quantum circuits," in *Proc. 42nd ACM SIGPLAN Int. Conf. Program. Lang. Design Implement. (PLDI)*, 2021, pp. 222–236, doi: 10.1145/3453483.3454040.

[31] E. Muñoz-Coreas and H. Thapliyal, "Quantum circuit design of a T-count optimized integer multiplier," *IEEE Trans. Comput.*, vol. 68, no. 5, pp. 729–739, May 2019, doi: 10.1109/TC.2018.2882774.

[32] R. Babbush *et al.*, "Encoding electronic spectra in quantum circuits with linear T complexity," *Phys. Rev. X*, vol. 8, no. 4, p. 041015, Oct. 2018, doi: 10.1103/PhysRevX.8.041015.

[33] A. Javadi-Abhari *et al.*, "Quantum computing with Qiskit," arXiv:2405.08810 [quant-ph], preprint, 2024.

[34] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Operating Syst. (ASPLOS)*, 2019, pp. 1001–1014, doi: 10.1145/3297858.3304023.

[35] B. Tan and J. Cong, "Optimality study of existing quantum computing layout synthesis tools," *IEEE Trans. Comput.*, vol. 70, no. 9, pp. 1363–1373, Sep. 2021, doi: 10.1109/TC.2020.3009140.

[36] S. Li, X. Zhou, and Y. Feng, "Qubit mapping based on subgraph isomorphism and filtered depth-limited search," *IEEE Trans. Comput.*, vol. 70, no. 11, pp. 1777–1788, Nov. 2021, doi: 10.1109/TC.2020.3023247.

[37] P. Zhu, W. Ding, L. Wei, X. Cheng, Z. Guan, and S. Feng, "A variation-aware quantum circuit mapping approach based on multi-agent cooperation," *IEEE Trans. Comput.*, vol. 72, no. 8, pp. 2237–2249, Aug. 2023, doi: 10.1109/TC.2023.3242208.

[38] C. Luo, S. Cao, S. Guo, and C. Hu, "Towards eective local search for qubit mapping," *IEEE Trans. Comput.*, vol. 74, no. 6, pp. 1897–1910, Jun. 2025, doi: 10.1109/TC.2025.3544869.

[39] K. Fang, M. Zhang, R. Shi, and Y. Li, "Dynamic quantum circuit compilation," *IEEE Trans. Comput.*, vol. 75, no. 2, pp. 748–759, Feb. 2026, doi: 10.1109/TC.2025.3643826.

[40] T. Satoh, Y. Ohkura, and R. Van Meter, "Subdivided phase oracle for NISQ search algorithms," *IEEE Trans. Quantum Eng.*, vol. 1, pp. 1–15, 2020, doi: 10.1109/TQE.2020.3012068.

[41] B. Pokharel and D. A. Lidar, "Better-than-classical Grover search via quantum error detection and suppression," *npj Quantum Inf.*, vol. 10, no. 1, p. 23, Feb. 2024, doi: 10.1038/s41534-023-00794-6.

[42] T. Lubinski *et al.*, "Application-oriented performance benchmarks for quantum computing," *IEEE Trans. Quantum Eng.*, vol. 4, pp. 1–32, 2023, doi: 10.1109/TQE.2023.3253761.

[43] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum amplitude amplification and estimation," in *Quantum Computation and Information*, ser. Contemporary Mathematics. Providence, RI, USA: American

Mathematical Society, 2002, vol. 305, pp. 53–74, doi: 10.1090/conm/305/05215.

[44] F. Arute *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, no. 7779, pp. 505–510, Oct. 2019, doi: 10.1038/s41586-019-1666-5.

[45] S. Imre, "Quantum existence testing and its application for finding extreme values in unsorted databases," *IEEE Trans. Comput.*, vol. 56, no. 5, pp. 706–710, May 2007, doi: 10.1109/TC.2007.1032.



Prasad Patil received his Bachelor's degree in Computer Science and Engineering from Shri Ramdeobaba Kamla Nehru Engineering College, Nagpur University, Maharashtra, India, in 2010. He began his career at the Tata Research Development and Design Centre (TRDDC), Pune, Maharashtra, India, in 2011. He joined TRDDC as a researcher, where he worked on various research projects and is the inventor of multiple patents. He later joined Tata Consultancy Services (TCS), Nagpur, Maharashtra, India, in 2022, where he has worked on large-scale enterprise systems, cloud migration, and advanced research-driven solutions. He is currently a Ph.D. scholar at the Indian Institute of Information Technology (IIIT) Nagpur, Maharashtra, India, and is an ACM India Anveshan Setu Fellow (2025) at IIT Hyderabad, Telangana, India. His current research focuses on quantum computing, deep learning, generative AI, and data mining.



Jitendra Tembhurne received the B.E. degree in Computer Technology from KITS, Ramtek, Nagpur University, Maharashtra, India, in 2003, and the M.E. degree in Computer Science and Engineering from MGM's College of Engineering, SRTMU Nanded, Maharashtra, India, in 2011. Ph.D. degree in Computer Science and Engineering from Visvesvaraya National Institute of Technology, Nagpur, Maharashtra, India, in 2017. From 2005 to 2011, he was an Assistant Professor with Computer Technology Department, KITS Ramtek. From 2016 to 2018, he was an Assistant Professor with the Computer Engineering Department, SVPCET, Nagpur, Maharashtra, India. Since 2018, he has been an Assistant Professor in Department of Computer Science and Engineering, Indian Institute of Information Technology (IIIT), Nagpur, Maharashtra, India. Also, author of 50+ articles published in international conferences and journals. His research interests include medical imaging, deep learning, information security, and quantum computing.



Suvra Jyoti Choudhury received the B.Tech. degree in Computer Science and Engineering from the West Bengal University of Technology, India, in 2009, and the M.Tech. degree in Information Technology from the Indian Institute of Engineering Science and Technology, Shibpur, India, in 2014. He received the Ph.D. degree in Computer Science from the Indian Statistical Institute, Kolkata, India, in 2022. He is currently an Assistant Professor at the Indian Institute of Information Technology, Nagpur, India. His research interests include image processing, artificial neural networks, and machine learning.